taoFPGA: Architecture, Implementation, and Evaluation of a Quantized Transformer Processing Core on Edge FPGA

Eliran Turgeman, Shay Rask

Faculty of Engineering, Bar-Ilan University

Academic Supervisor: Dr. Leonid Yavits | Project Mentor: David Freud

***Abstract*—**Transformer models are increasingly deployed on power- and resource-constrained edge devices, where their quadratic attention cost and large dense matrix multiplications strain both compute and memory bandwidth. This paper presents taoFPGA, a quantized INT8 transformer processing core built around a 16×16 weight-stationary systolic array fused with dedicated fixed-point Softmax and GELU engines, verified end to end from RTL simulation through physical signoff and real hardware bring-up on a Xilinx Zynq-7020 (PYNQ-Z2). We detail a tolerance-bounded verification methodology carried consistently from SystemVerilog testbenches through a Python/NumPy golden model validated against physical silicon, a synthesis-level DSP-inference defect whose correction reduced LUT utilization by 49.3% and total on-chip power by 3.1%, and a full-scale hardware validation run matching simulation to zero error across 32,000 output elements. We further report a kernel-level benchmark against a real, numerically validated ViT-Tiny software baseline, achieving up to 97.73× measured speedup (70.02× and 97.73× across the two benchmarked kernel shapes) over the board's ARM Cortex-A9, corresponding to a total energy efficiency of 2.25 GOPS/W (2.47 GOPS/W on dynamic power alone) at peak throughput — while explicitly analyzing why the accelerator's fixed pipeline does not structurally correspond to any single Vision Transformer operation, a distinction we treat as a methodological contribution in its own right. We close with a quantified account of the design's remaining hardware constraints, principally a 64 KB single-transfer DMA ceiling, and the Scatter-Gather DMA redesign identified as its prerequisite fix.

***Index Terms*—**FPGA, transformer accelerator, systolic array, vision transformer, quantization, Softmax, GELU, Zynq, PYNQ, hardware/software co-design.

I. INTRODUCTION & MOTIVATION

A. The Edge AI Compute Dilemma

Transformer models now dominate both natural-language processing and computer vision, but their computational profile — quadratic self-attention and large dense projections in every encoder layer — sits uneasily on embedded edge platforms, where DRAM bandwidth, on-chip memory, and power budget are all scarce simultaneously [1]. Prior FPGA accelerators for transformer-family models converge on a small set of recurring design choices — systolic or array-of-PE matrix multiplication, fixed-point quantization, and dedicated non-linear activation units — to close this gap [2]–[5]. This project targets that same design space on a resource-constrained, commodity Zynq-7020 SoC.

B. Design Intent & Methodology

taoFPGA was developed bottom-up: each arithmetic core — the systolic matrix multiplier, the Softmax normalization engine, and the GELU activation unit — was designed and verified in isolation against a real-valued golden reference before being composed into a single streaming pipeline (MatMul → Softmax → GELU), integrated into a complete SoC, physically implemented, and finally evaluated against real Vision Transformer (ViT) workload shapes on physical hardware. A recurring methodological principle, arrived at empirically over the course of the project, governs every phase: no layer of the system — golden model, integration script, board, register map, or DMA transfer — may be assumed correct on the strength of another layer's correctness. Each was checked directly against its own ground truth.

C. Key Contributions & Paper Outline

(1) A 16×16 INT8 weight-stationary systolic array fused with dedicated fixed-point Softmax and GELU engines into a single streaming pipeline.

(2) A tolerance-bounded verification methodology carried consistently from SystemVerilog RTL simulation through a Python/NumPy golden model validated against physical silicon.

(3) Identification and correction of a synthesis-level DSP-inference defect, reducing LUT utilization by 49.3% and total on-chip power by 3.1% at signoff.

(4) First physical hardware validation of the design on a PYNQ-Z2, matching simulation to zero error across 32,000 output elements at full scale.

(5) A kernel-level hardware/software benchmark against a real, numerically validated ViT-Tiny baseline, reporting 70.02×-97.73× measured speedup and a total energy efficiency of 2.25 GOPS/W at peak throughput, together with an explicit architectural analysis of why the accelerator's fixed pipeline does not map onto any single ViT operation.

(6) A quantified account of the design's open hardware constraints — principally a 64 KB DMA transfer ceiling — and the re-synthesis path identified to resolve them.

The remainder of this paper is organized as follows. Section II details the core hardware architecture and its fixed-point formulation. Section III covers RTL verification. Section IV describes SoC integration on the PYNQ-Z2. Section V reports physical implementation and signoff. Section VI presents experimental evaluation and hardware bring-up. Section VII discusses remaining bottlenecks and future work, and Section VIII concludes.

II. CORE HARDWARE ARCHITECTURE & MATHEMATICAL FORMULATION

A. Systolic Matrix Multiplication Core

The matrix multiplication core (MM_ultra) is a weight-stationary 16×16 systolic array of INT8 processing elements (256 MACs total), fed by dedicated feature and weight input buffers (MM_in_buffer, MM_buffer) that stream operands into the array in A_size-wide parallel beats. Runtime-configurable block-count registers (F_width_block_num, W_width_block_num) parameterize the contraction and output dimensions, both constrained to multiples of the array width. Accumulated products are rounded and saturated to INT8 by a dedicated shifter: the design rounds half-up on an arithmetic right shift — equivalently, add 2^(shift−1) then shift, or inspect the most-significant discarded bit — before clamping to [−128, 127]. Both formulations were confirmed mathematically identical during hardware bring-up (Section VI) when the software golden model was cross-checked line-by-line against this shifter's RTL. Formally, for a pre-shift accumulator value z and a runtime-configurable shift $\in \mathbb{Z}^+$, the quantization operator realized in hardware is

$$\mathrm{Quant}(z) = \mathrm{clip}\left(\left\lfloor (z + 2^{\,\mathrm{shift}-1}) \cdot 2^{-\mathrm{shift}} \right\rfloor, -128,\ 127\right) \quad (1)$$

where $\lfloor\cdot\rfloor$ denotes floor (equivalently, arithmetic right shift for two's-complement operands) and clip(·, lo, hi) saturates its argument to [lo, hi]. The degenerate case shift = 0 is handled as a direct clip of z with no rounding term. Listing 1 excerpts the actual RTL realization of (1) in right_shifter.v, which computes the round term by inspecting the discarded bit directly rather than via an explicit add-then-shift, the two formulations having been confirmed identical as noted above.

```
// right_shifter.v -- round-half-up + saturate to INT8
assign temp1_out = data_in >>> shift;
assign temp2_out =
    data_in[shift-1] ? temp1_out + 1 : temp1_out;
// under_min/over_max: range checks on temp2_out
case ({under_min, over_max})
    2'b10: data_out = {1'b1, {(W-1){1'b0}}}; // -128
    2'b01: data_out = {1'b0, {(W-1){1'b1}}}; // +127
    default: data_out = temp2_out[W-1:0];
endcase
```

Listing 1. Round-half-up and saturate operator, right_shifter.v (Eq. 1).

B. Fixed-Point Non-Linear Arithmetic Engines

Softmax normalization (Softmax_control / Softmax) is a three-pass scalar architecture operating on one buffered row at a time: pass one finds the row maximum via a running comparator; pass two streams the row a second time, computing and accumulating exp(x−max) via a base-2 exponential approximation (a rational scaling of x by $\log_2(e)$, split into integer and fractional components with linear interpolation of the fractional power-of-two term); pass three streams the row a third time, computing the final normalized value exp(x−max−ln(Σ)) using a second instance of the same exponential unit combined with a piecewise logarithm approximation (Ln_module) to avoid an explicit division. Formally, for a row x with elements x_i, the three passes compute

$$m = \max_i(x_i)$$
$$S = \sum_i 2^{(x_i - m)\log_2 e}$$
$$y_i = 2^{((x_i - m) - \ln S)\log_2 e} \quad (2)$$

an exact base-2 reformulation of $\mathrm{softmax}(x)_i = \exp(x_i - m)/\Sigma$ that matches the hardware's own $\log_2(e)$-scaled exponential and Ln_module log-domain division-avoidance strategy exactly, rather than the textbook exp/ln formulation alone. Fig. 1 depicts the resulting finite-state control flow: each of the three passes streams the full N-element row exactly once, strictly sequentially, for a total latency of 3N cycles per row — the structural origin of the serial Softmax bottleneck quantified in Section VII.B.

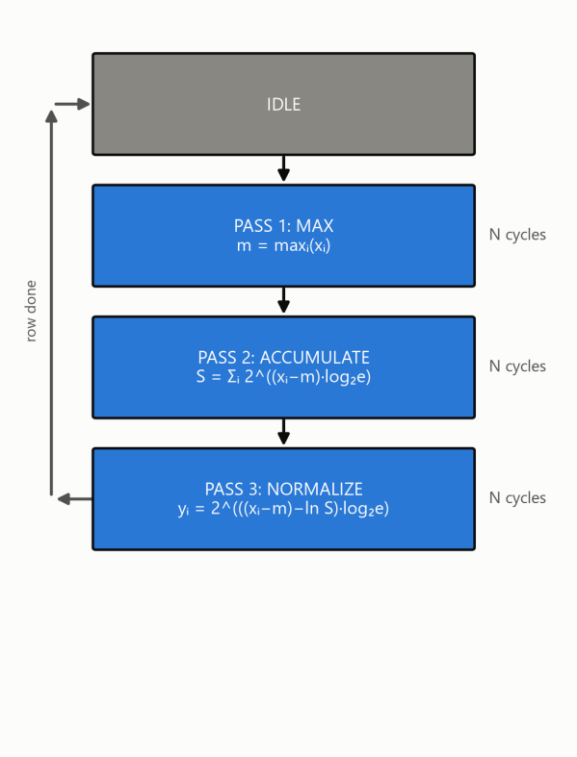

Fig. 1. Softmax_control's 3-pass FSM and its per-pass equations (Eq. 2).

The GELU activation (gelu.v) is a two-segment piecewise-linear approximation: inputs beyond a fixed saturation threshold pass through unmodified or clamp to zero according to sign, and inputs within that range are computed via a linear correction term generated by a dedicated interpolation submodule (lin.v).

C. Numerical Quantization Strategy

Features, weights, and pipeline output are all INT8. The matmul accumulates in extended precision before the shift-and-saturate step described above; Softmax and GELU each carry independent runtime-configurable fixed-point scale registers (softmax_scale_in, softmax_scale_out, gelu_scale), with softmax_scale_out and gelu_scale constrained equal so that the two activation stages interpret the shared int8 boundary between them consistently.

III. RTL VERIFICATION & BEHAVIORAL SIMULATION (CADENCE XCELIUM)

A. Testbench Architecture & Golden Reference Model

Four SystemVerilog testbenches (gelu_tb, Softmax_top_tb, MM_Ultra_tb, and an integration-level transformer_block_tb) each implement a real-valued ($real) golden reference directly in SystemVerilog: an integer matmul with the same round-half-up-and-saturate rule as the RTL shifter, a numerically stable softmax via $exp, and a tanh-approximation GELU via $tanh. The integration testbench chains all three references to validate the full streaming pipeline against one coherent golden model.

B. Simulation Results & Numerical Precision Verification

Verification is tolerance-bounded throughout, not exact-match: unit-level tests compare against a scale-dependent LSB threshold, and the full-pipeline integration test uses a 6-LSB tolerance at the shared Softmax-output/GELU scale to absorb error compounding across the two INT8 quantization boundaries. At full scale (200×96×160), the integration run recorded zero of 32,000 output elements outside tolerance, with an 18,083-cycle end-to-end pipeline latency.

C. Pre-Silicon Logic Debugging

Three representative issues surfaced during simulation bring-up: a stimulus-side forward-reference bug in a testbench control signal; an Xcelium elaboration-time memory blowup traced to a generate-loop-unrolled per-scalar wire array that scaled combinatorially with matrix size, resolved by replacing it with an on-demand packing function; and a datapath port declared as output reg rather than output wire, which was silently dropped during elaboration rather than flagged as an error.

IV. SOC INTEGRATION & HARDWARE PLATFORM DESIGN (PYNQ-Z2 / ZYNQ-7020)

A. SoC Architecture

The accelerator is wrapped as an AXI4-Lite-controlled, dual AXI4-Stream-slave / single AXI4-Stream-master peripheral (transformer_block_axi_top) and integrated alongside the Zynq-7020's dual-core ARM Cortex-A9 processing system, an AXI SmartConnect interconnect, and three independent AXI Direct Memory Access engines (feature, weight, and result channels) via Xilinx IP Integrator. Fig. 2 shows the complete top-level datapath: the PS7 configures the pipeline over AXI-Lite and streams feature and weight tensors in via two MM2S DMA channels, while a third, independent S2MM channel returns the GELU stage's output; internally, width-adapter glue logic bridges each stage's differing datapath width, exactly as detailed in Section IV.B.

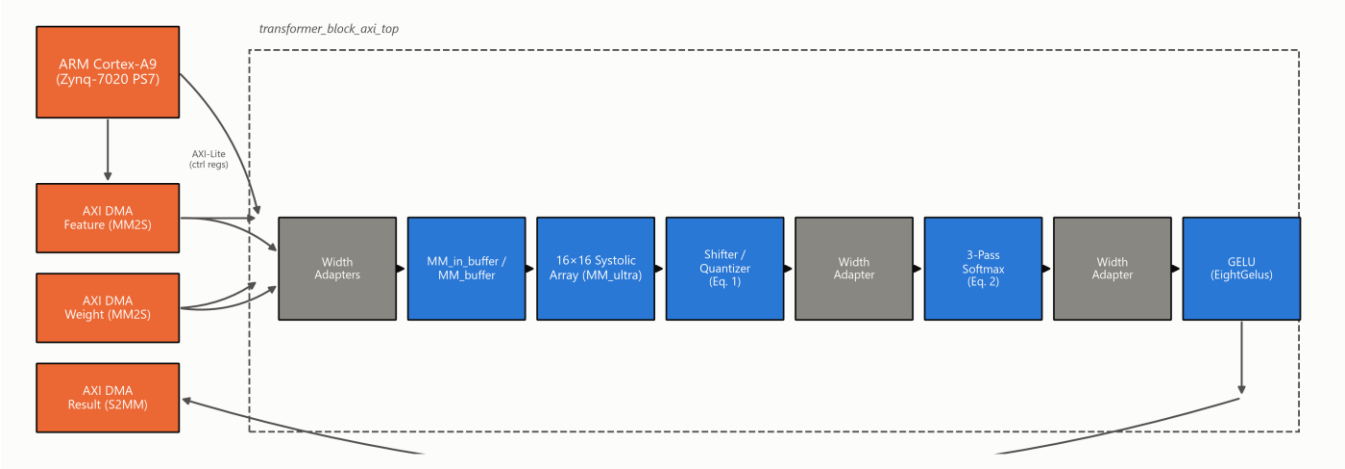

Fig. 2. Complete top-level SoC architecture: PS7, three AXI DMA channels, and the transformer_block_axi_top internal pipeline.

B. Physical Adaptations for Board Deployment

Integration surfaced two categories of adaptation beyond the standalone core. First, a synthesis-time design rule check flagged over 40 instances of block-RAM control logic driven by asynchronously reset registers inside the input/output buffers — a pattern Xilinx's own methodology documentation warns can corrupt memory contents outside static timing analysis' visibility. This was resolved across twelve RTL files by converting every affected sensitivity list from asynchronous to synchronous reset. Second, bridging the array's A_size-wide parallel datapath to Softmax's serial, one-scalar-per-cycle interface (and back to GELU's narrower parallel lanes) required bespoke AXI4-Stream width converters and a purpose-built row-boundary signal, since the matmul core's own frame-completion signal marks only the end of the entire matrix, not each row.

C. Software Infrastructure for Hardware Bring-Up

To extend the tolerance-bounded methodology of Section III to physical silicon, the SystemVerilog golden references were ported to a bit-faithful Python/NumPy implementation, cross-checked against the RTL shifter's exact rounding behavior rather than assumed equivalent, and used to validate hardware output read back over the board's PYNQ Python driver.

V. PHYSICAL IMPLEMENTATION, P&R, AND SIGN-OFF OPTIMIZATIONS

A. Baseline Physical Sign-Off

The first complete post-route signoff of the full SoC closed timing at the 100 MHz target with a Worst Negative Slack (WNS) of +0.361 ns and a Vivado-estimated total on-chip power of 1.741 W (1.590 W dynamic, 0.151 W static).

B. The DSP Inference Gap & Attribute Resolution

Post-route utilization at this baseline was LUT-bound rather than DSP-bound: only 13 of 220 available DSP48E1 slices (5.91%) were in use while Slice LUTs sat at 56.91%, despite the 256-MAC systolic array being the design's dominant arithmetic structure — a strong signal that the array's multiply-accumulate operations were being synthesized into general LUT/CARRY4 fabric instead of dedicated DSP hard macros. Root-causing traced this to a synthesis attribute scoped at the wrong granularity relative to Vivado's register-level DSP inference rules (cf. Xilinx UG901 [6]): as Listing 2 shows, attaching use_dsp to the enclosing always block is a silent no-op — synthesis produced byte-identical LUT/DSP/CARRY4 counts regardless of its value — whereas attaching it to the register declaration that actually holds the multiply-accumulate result is what Vivado's inference pass recognizes.

```
// PE.v -- BEFORE: attribute on the always block (no-op)
(* use_dsp = "yes" *)
always @(posedge clk) begin
    psum_out <= psum_in + x_in*reg_w;
end

// AFTER: attribute on the register declaration (per
UG901)
(* use_dsp = "yes" *) reg signed
    [2*data_width+log2_array_m-1:0] psum_out_r;
always @(posedge clk)
    psum_out_r <= psum_in + x_in*reg_w;
```

Listing 2. The UG901 DSP-inference attribute-placement fix, PE.v.

Correcting the attribute placement alone would map all 256 PEs to DSP48E1 hard macros — more than the xc7z020's 220-slice budget once Softmax and GELU's own arithmetic (≈13 DSPs) is accounted for. PE_array.v therefore parameterizes the split per PE row (NUM_DSP_ROWS = 12 of 16), mapping

exactly 192 of 256 PEs to DSP48E1 and leaving the remaining 4 rows (64 PEs) LUT/CARRY4-mapped — a deliberate, budget-sized partial mapping, not an all-or-nothing switch. This reduced full-SoC post-route Slice LUTs by 49.3% (30,276 → 15,363) and CARRY4 primitives by 61.8% (4,874 → 1,862), brought DSP48E1 usage to 205 of 220 (192 from the array plus Softmax/GELU's ≈13), and reduced total on-chip power by 3.1% to 1.687 W.

C. Physical Congestion vs. Timing Trade-Off

The same fix concentrated 205 of 220 DSP48E1 slices (93.18% of the device's entire DSP column capacity, 15 slices of headroom remaining) into a much denser physical footprint than the previous LUT-diffuse implementation. Post-route WNS correspondingly tightened from +0.361 ns to +0.293 ns (≈19% less margin, timing still met) — an expected, explicitly-anticipated trade-off between area/power efficiency and routing congestion around a densely packed DSP column, not a regression.

D. Bitstream & Hardware Platform Sign-Off

The signed-off, DSP-corrected checkpoint passed pre-bitstream design rule checking with zero errors and generated a bitstream with zero warnings and zero critical warnings, exported as design.bit and a fixed hardware platform archive (design.xsa) for downstream software and PYNQ deployment.

TABLE I. POST-ROUTE SIGNOFF METRICS, BEFORE/AFTER THE DSP-INFERENCE FIX

Metric	Before fix	After fix	Δ
Slice LUTs (post-route, full SoC)	30,276 / 53,200 (56.91%)	15,363 / 53,200 (28.88%)	−49.3%
CARRY4 (post-route, full SoC)	4,874 / 13,300	1,862 / 13,300	−61.8%
DSP48E1	13 / 220 (5.91%)	205 / 220 (93.18%)	+192
WNS @ 100 MHz	+0.361 ns	+0.293 ns	−19% margin
Total on-chip power	1.741 W	1.687 W	−3.1%

VI. EXPERIMENTAL EVALUATION & HARDWARE BRING-UP

A. Benchmarking Methodology

Hardware kernel throughput was benchmarked on the physical PYNQ-Z2 against two matrix shapes representative of ViT-Tiny's real layer dimensions (embedding dimension 192, sequence length 197): a 192×192 projection-sized kernel and a 192×320 MLP-shaped kernel, each compared against an equivalent computation of the identical fused matmul-Softmax-GELU operation executed on the board's ARM Cortex-A9.

B. Performance Metrics & Acceleration

The hardware kernel achieved 2.487 GOP/s and 3.795 GOP/s on the two benchmarked shapes respectively, against 0.036 and 0.039 GOP/s on the CPU — measured speedups of 70.02× and 97.73× for the identical operation. At the 3.795 GOP/s peak (192×320, MLP-shaped kernel), against the post-route signoff power figures of Table I (1.687 W total on-chip, 1.538 W dynamic), the design achieves a total energy efficiency of 2.25 GOPS/W and a dynamic energy efficiency of 2.47 GOPS/W. These efficiency figures combine a live-measured kernel throughput with Vivado's post-route, activity-based power estimate for the full SoC design — not a physically instrumented power reading synchronized to the benchmark run itself — and should be read as a signoff-grade estimate accordingly. Figs. 3–5 present latency, throughput, and speedup for both shapes.

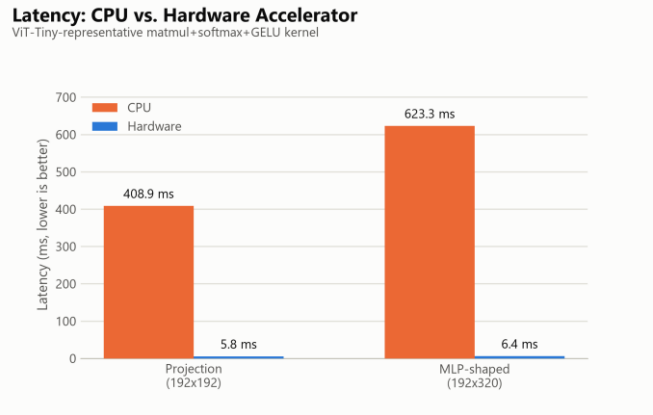

Fig. 3. Latency, CPU vs. hardware, both benchmarked shapes.

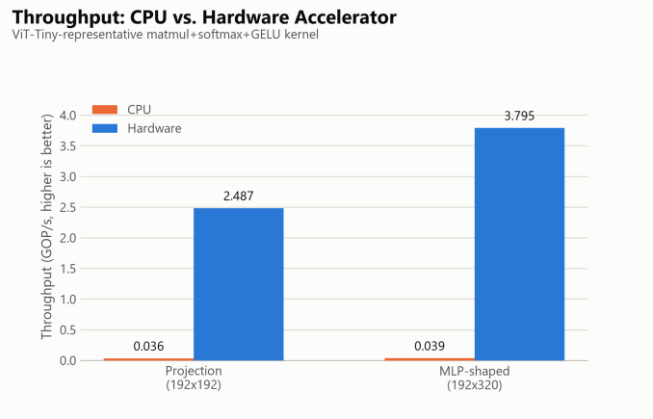

Fig. 4. Throughput (GOP/s), CPU vs. hardware, both shapes.

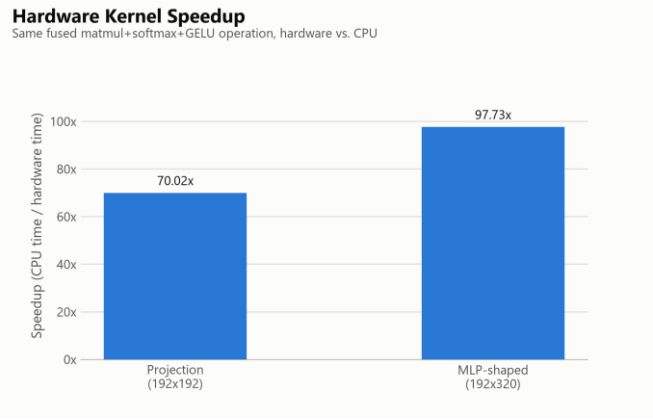

Fig. 5. Measured hardware kernel speedup over CPU.

TABLE II. KERNEL BENCHMARK RESULTS, PYNQ-Z2 vs. ARM CORTEX-A9

Shape	HW latency	CPU latency	Speedup
192×192 (projection)	5.84 ms	408.9 ms	70.02×
192×320 (MLP-shaped)	6.38 ms	623.3 ms	97.73×

C. Physical Silicon Verification

Independent of the kernel benchmark above, the full pipeline was validated at the exact scale exercised in Section III's simulation (200×96×160, an 18,083-cycle simulated end-to-end latency): the physical hardware run reproduced the software golden model with zero of 32,000 output elements outside the established tolerance bound, confirming that behavioral simulation and physical silicon agree at the design's originally verified scale.

D. Workload Characterization vs. ViT Networks

A software-only ViT-Tiny (patch16, 224×224, 5.7M parameters [7], [8]) forward pass, reimplemented from scratch in NumPy and validated against its reference PyTorch implementation to a maximum logit difference of 1.02×10^{-5}, reproduced an identical top-1 prediction with 87.26% confidence on a sample image — a single-sample confidence score, not a classification accuracy measured over a labeled evaluation set. Critically, the accelerator's fixed matmul→Softmax→GELU pipeline does not structurally correspond to any single ViT-Tiny or ViT-Base operation: multi-head self-attention consists of matrix multiplications and one Softmax with no GELU stage, while the MLP block consists of a matrix multiplication and GELU with no Softmax between them. Substituting the accelerator into either would silently apply an activation or normalization step the real operation does not include, corrupting the result. This architectural mismatch — verified directly against the RTL rather than assumed — is the reason Section VI.A–B report a kernel-level throughput benchmark rather than an end-to-end accelerated ViT inference claim.

E. Hardware Bring-Up Observations & Edge Cases

Two open hardware findings emerged during bring-up. First, a root-caused defect in the input buffer's row-replay address counter (MM_in_buffer): for a single-row configuration (F_length = 1), the counter advances to 1 immediately on start but its wrap-to-zero condition checks against F_length−1 = 0, a value it can then never reach again, causing the buffer to read stale data indefinitely. Second, a related but distinct DMA hang observed at a two-row configuration was traced far enough to rule out the same cause but not far enough to identify its root; both are confined to matrix shapes far smaller than any configuration this design was validated at (Section III.B, VI.C), and the codebase was frozen with both documented as open items (Section VII) rather than resolved in this scope.

VII. ENGINEERING DISCUSSION, BOTTLENECKS & FUTURE WORK

A. DMA Buffer Boundary Constraints

The design's three AXI DMA engines were synthesized with a 16-bit transfer-length register, capping any single DMA transfer at 65,536 bytes. This directly prevented benchmarking the real 768-wide ViT-Tiny MLP hidden dimension (a 192×768 weight buffer alone requires 147,456 bytes) and, more broadly, blocks batch processing entirely: streaming even two images' feature matrices against a shared weight matrix already overflows the ceiling for every benchmarked shape in this work. Splitting a transfer across multiple smaller DMA calls is not a safe software workaround with the current RTL, since the input buffer's write-address counters reset on the stream's tlast signal, which the design's direct-register DMA mode asserts at the end of every transfer issued — not only a genuinely final one — corrupting rather than merely truncating a chunked transfer.

B. Serial Softmax Throughput Bottleneck

The systolic array produces A_size parallel INT8 results per cycle, while the Softmax engine consumes and produces exactly one scalar per cycle across three full passes over each row. This structural mismatch — a fully parallel two-dimensional compute stage feeding a one-dimensional serial normalization stage — is a standing throughput bottleneck independent of clock frequency or DSP utilization, and is the most direct architectural target for a future parallel or pipelined Softmax redesign.

C. Architectural Roadmap

Two concrete next steps follow directly from Sections VII.A–B: (i) replacing direct-register DMA with genuine Scatter-Gather DMA (or, more simply, re-synthesizing with a wider transfer-length register), giving explicit per-descriptor frame-boundary control and removing the single-transfer ceiling that currently blocks both full-width MLP evaluation and batch processing; and (ii) decoupling the matmul, Softmax, and GELU stages into independent streaming nodes with real backpressure support, addressing both the serial Softmax bottleneck and a separately documented limitation in the GELU stage's lack of an internal skid buffer.

VIII. CONCLUSION

taoFPGA demonstrates a complete hardware/software co-design lifecycle for a quantized transformer processing core, from fixed-point mathematical formulation and tolerance-bounded RTL verification through physical signoff, real hardware bring-up, and a kernel-level benchmark against a genuinely validated software ViT baseline. Beyond the measured results — a 49.3% LUT reduction and 3.1% power reduction from a single synthesis-level fix, zero-error silicon validation at full verified scale, and up to 97.73× kernel speedup over an embedded ARM CPU — the project's governing discipline throughout was to verify every layer of the system against its own ground truth rather than inherit correctness from an adjacent layer, and to report every limitation — architectural, numerical, or a hardware transfer ceiling — as precisely as the result being claimed. The 64 KB DMA ceiling and its Scatter-Gather resolution path, together with the serial Softmax bottleneck, define a clear and quantified roadmap for the next iteration of this work.

REFERENCES

[1] A. Vaswani et al., “Attention Is All You Need,” in Proc. NeurIPS, 2017.

[2] Y. Zhu, Q. Li, S. Chen, and Y. Ma, “High-Performance FPGA Acceleration for Transformer-Based Models: A Survey,” IEEE Trans. VLSI Syst., 2023.

[3] Z. Li, H. Chen, C. Cheng, and X. Huang, “ME-ViT: A Single-Load Memory-Efficient FPGA Accelerator for Vision Transformers,” in Proc. ACM/IEEE ISLPED, 2022.

[4] H. Liu and J. Sun, “FTRANS: Energy-Efficient Acceleration of Transformers using FPGA,” in Proc. ACM/SIGDA FPGA Symp., 2021.

[5] Y. Cai, A. Zhang, and X. Chen, “A High-Throughput and Energy-Efficient FPGA-Based Transformer Accelerator,” in Proc. IEEE IPDPS, 2020.

[6] AMD/Xilinx, “Vivado Design Suite User Guide: Synthesis (UG901),” AMD, San Jose, CA, USA.

[7] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in Proc. ICLR, 2021.

[8] R. Wightman, “PyTorch Image Models (timm),” GitHub repository, 2019. [Online]. Available: https://github.com/huggingface/pytorch-image-models